

PROJECT DRAFT REPORT (PHASE 1)

AI Codebase Intelligence System using GraphRAG and Large Language Models

Abstract

This project proposes an AI-powered platform capable of understanding complete software repositories rather than individual files. By combining source-code parsing, graph representations, vector databases, Retrieval-Augmented Generation (RAG), and Large Language Models, the system assists developers in exploring architecture, understanding dependencies, reviewing code, and generating documentation. Phase 1 focuses on repository parsing, dependency extraction, and architecture modeling.

1. Introduction

Large software repositories are difficult to understand due to thousands of interconnected files. New developers spend considerable time understanding architecture before contributing. The proposed solution automates repository understanding using modern AI techniques and creates a semantic knowledge base for future intelligent querying.

2. Problem Statement

Existing static analysis tools provide syntax-level insights but lack semantic reasoning and conversational interfaces. Developers still manually search code, documentation, and dependencies. This project aims to create an intelligent assistant capable of repository-wide reasoning.

3. Objectives

- Clone and analyze GitHub repositories.
- Parse source code using Tree-sitter.
- Extract functions, classes, imports, and dependencies.
- Generate repository dependency graphs.
- Prepare a knowledge base for AI reasoning in later phases.

4. Existing vs Proposed System

Existing: IDE search, documentation, and static analyzers provide fragmented understanding.
Proposed: Unified AI system with repository parsing, graph modeling, semantic search, and conversational explanations.

5. Phase-wise Methodology

Phase 1: Repository cloning, AST parsing, dependency extraction, architecture graph generation.

Phase 2: Embeddings, ChromaDB/FAISS, GraphRAG, semantic code search, LLM-based Q&A;

Phase 3: AI code review, bug detection, documentation generation, UML creation, refactoring suggestions, deployment.

6. System Architecture

GitHub Repository → Git Clone Module → Tree-sitter Parser → AST & Metadata Extraction → Dependency Graph → Knowledge Graph → Vector Database → LLM Engine → Web Interface.

7. Technology Stack

Python, FastAPI, Tree-sitter, GitPython, NetworkX, Neo4j (future), Sentence Transformers, ChromaDB/FAISS, Gemini/OpenAI API, React/Streamlit, Docker, GitHub Actions.

Phase	Major Deliverable
Phase 1	Repository Parsing & Dependency Graph
Phase 2	RAG-based Code Understanding
Phase 3	AI Code Review & Documentation

8. Deliverables of Phase 1

Repository parser, AST extractor, dependency graph generator, architecture design, GitHub repository structure, and documentation.

9. Tentative Timeline

- Week 1: Literature Survey & Setup
- Week 2: Repository Cloning
- Week 3: Code Parsing
- Week 4: Dependency Graph & Documentation

10. Expected Outcomes

The system will reduce repository onboarding time, automate repository understanding, and provide a scalable AI foundation for intelligent software engineering.

11. Future Scope

GraphRAG with Neo4j, pull request reviews, vulnerability detection, automatic test generation, CI/CD integration, and enterprise-scale repository analytics.

12. References

[1] Attention Is All You Need (2017)
[2] Retrieval-Augmented Generation (2020)
[3] Tree-sitter Documentation
[4] FastAPI Documentation
[5] LangChain Documentation

13. Conclusion

This project combines program analysis and modern AI to build an intelligent software engineering assistant. Phase 1 establishes the core repository understanding pipeline.